AgentZero — Implementation Plan

Agent-less Windows System Vulnerability & Network Scanner

> **Organisation:** National Technical Research Organisation (NTRO) **Domain:** Blockchain & Cybersecurity | **Category:** Software **Repo:** AYUSH-KAVYA/agentZero

Executive Summary

AgentZero is an on-demand, agent-less desktop application that audits a Windows 10/11 system for misconfigurations and network-level vulnerabilities. It queries the OS using native WMI, PowerShell, and Registry APIs, cross-references findings against a CVE/CIS benchmark database, crawls for open-source exploits and patches, and produces consolidated PDF/HTML reports — all without installing a persistent background agent.

Proposed Changes to Your 6-Step Blueprint

> IMPORTANT
>
> Your 6-step plan is solid. Below are **targeted refinements** — not overhauls — to strengthen the architecture.

Change 1: Split Python Engine into 3 Distinct Modules (Step 2)

Your plan: Build modular Python CLI utilities. **Refinement:** Formalize these into **3 independent sub-packages** that can be invoked and tested in isolation:

Module	Responsibility
`core/system_scanner/`	All 25+ system-level checks (WMI, Registry, Defender, AMSI, etc.)
`core/network_scanner/`	ARP, DNS, TCP/UDP, RPC, LLDP/CDP, VLAN mapping
`core/vuln_engine/`	CVE cross-referencing, CIS benchmarking, exploit/patch crawling

Why: A monolithic `scanner.py` becomes unmaintainable at ~25 system checks + ~7 network checks + vulnerability mapping. Separate packages allow parallel development and isolated PyInstaller testing.

Change 2: Add an Exploit & Patch Crawler (New — Required by PS92)

Your plan: Acquire a lightweight CVE dataset and cross-reference. **Refinement:** PS92 explicitly requires the tool to **"search and crawl for related available open-source exploits and their patches."** This is more than static lookup. Add a `vuln_engine/exploit_crawler.py` that:

- Queries the **NVD API** (NIST) for CVE details matching detected software versions
- Scrapes **Exploit-DB** and **GitHub Advisory Database** for matching public exploits
- Maps each CVE to its **Microsoft KB patch** and checks if the hotfix is installed
- Works offline-first with a bundled JSON snapshot, online-enhanced when connectivity exists

Change 3: Network Diagram Generation (New — Required by PS92)

Your plan: Map connected VLAN devices and identify open RPC endpoints. **Refinement:** PS92 explicitly requires **"the capability to formulate network diagram using these information."** Add a `network_scanner/diagram_generator.py` that:

- Takes the discovered topology (interfaces, LLDP/CDP neighbors, VLAN devices) as input
- Generates an interactive network diagram using **D3.js** (rendered in Electron) or a static SVG/PNG using **graphviz/pydot**
- Embeds the diagram in the HTML/PDF report

Change 4: Report Engine (Strengthened — Step 5)

Your plan: Parse JSON to populate a visual dashboard. **Refinement:** Add a dedicated report generation layer that produces:

- **HTML report** — Self-contained single-file HTML with embedded CSS/JS, interactive charts, and the network diagram

- **PDF report —** Generated from the HTML via Puppeteer (already bundled with Electron) or `weasyprint`
- **Severity scoring —** Each finding tagged with CVSS-aligned severity (Critical/High/Medium/Low/Info)
- **Remediation guidance —** Each vulnerability linked to its specific Microsoft KB article or CIS fix

Change 5: UAC Elevation Handling (New — Critical)

Not in your plan. The application needs admin privileges for:

- Reading HKLM registry keys (Defender exclusions, Sysmon config, UAC policies)
- WMI queries for AV, hotfixes, DNS cache
- Raw socket access for network scanning

Add UAC elevation logic in the Electron main process using `electron-sudo` or a custom manifest with `requireAdministrator`.

Change 6: AV/EDR Evasion-Friendly Design (New — Required by PS92)

Not in your plan. PS92 requires the solution to be **"AV/EDR friendly."** This means:

- No shellcode injection, no process hollowing, no memory patching
- All binaries must be signed (or at minimum, not trigger heuristic alerts)
- PowerShell commands use `-ExecutionPolicy Bypass` only for the current scope
- PyInstaller `.exe` should be packed with `--clean` and code-signed if possible
- Document known AV false-positive mitigation strategies

Revised 6-Step Implementation Plan

Step 1: Project Scaffolding & Dev Environment

Duration: Day 1-2

```
agentZero/
├── README.md
├── LICENSE
├── .gitignore
├── package.json                  # Electron + React
├── electron/
│   ├── main.js                   # Electron main process
│   ├── preload.js                # contextBridge IPC
│   └── ipc-handlers.js           # IPC handler registry
├── src/                          # React frontend
│   ├── App.jsx
│   ├── index.jsx
│   ├── components/
│   │   ├── Dashboard/
│   │   ├── SystemScan/
│   │   ├── NetworkScan/
│   │   ├── VulnReport/
│   │   └── NetworkDiagram/
│   ├── hooks/
│   ├── context/
│   └── styles/
├── python/                       # Python backend
│   ├── requirements.txt
│   ├── main.py                   # CLI entry point
│   ├── core/
│   │   ├── __init__.py
│   │   ├── system_scanner/
│   │   │   ├── __init__.py
│   │   │   ├── os_info.py
│   │   │   ├── dotnet_versions.py
│   │   │   ├── amsi_providers.py
│   │   │   ├── antivirus.py
│   │   │   ├── audit_policy.py
│   │   │   ├── autorun.py
│   │   │   ├── firewall.py
│   │   │   ├── defender.py
│   │   │   ├── certificates.py
│   │   │   ├── environment.py
│   │   │   ├── file_listing.py
│   │   │   ├── file_info.py
│   │   │   ├── hotfixes.py
│   │   │   ├── installed_products.py
│   │   │   ├── group_policy.py
│   │   │   ├── local_groups.py
│   │   │   ├── local_users.py
```

```
│   │   │   ├── ms_updates.py
│   │   │   ├── ntlm_settings.py
│   │   │   ├── rdp_connections.py
│   │   │   ├── rdp_sessions.py
│   │   │   ├── rdp_settings.py
│   │   │   ├── secure_boot.py
│   │   │   ├── sysmon.py
│   │   │   ├── uac_policies.py
│   │   │   ├── powershell_history.py
│   │   │   └── runner.py         # Orchestrates all system checks
│   │   ├── network_scanner/
│   │   │   ├── __init__.py
│   │   │   ├── arp_table.py
│   │   │   ├── dns_cache.py
│   │   │   ├── network_profiles.py
│   │   │   ├── network_shares.py
│   │   │   ├── connections.py
│   │   │   ├── rpc_endpoints.py
│   │   │   ├── open_ports.py
│   │   │   ├── interfaces.py
│   │   │   ├── lldp_cdp.py
│   │   │   ├── vlan_devices.py
│   │   │   ├── diagram_generator.py
│   │   │   └── runner.py
│   │   └── vuln_engine/
│   │       ├── __init__.py
│   │       ├── cve_database.py
│   │       ├── cis_benchmarks.py
│   │       ├── exploit_crawler.py
│   │       ├── patch_checker.py
│   │       ├── severity_scorer.py
│   │       └── runner.py
│   ├── report/
│   │   ├── __init__.py
│   │   ├── html_generator.py
│   │   ├── pdf_generator.py
│   │   └── templates/
│   │       ├── report_template.html
│   │       └── styles.css
│   └── data/
│       ├── cve_snapshot.json      # Offline CVE database
│       └── cis_windows_benchmarks.json
├── assets/                        # Icons, logos
├── scripts/
│   ├── build-python.sh            # PyInstaller build script
│   └── build-electron.sh          # electron-builder script
```

```
└── docs/
    └── architecture.md
```

Tasks:

- ✓
  Initialize git repo + remote
- Create the full directory structure
- Initialize `package.json` (Electron 28+, React 18, Vite)
- Initialize Python `requirements.txt` (wmi, scapy, psutil, python-nmap, requests, pydot, jinja2, weasyprint)
- Create `.gitignore` (node_modules, dist, build, **pycache**, *.exe, venv)
- Create `README.md` with project overview
- Set up ESLint + Prettier for JS, Black + Ruff for Python

Step 2: Python Extraction Engine — System Scanner

Duration: Day 3-7

Build all 25 system-level scanner modules. Each module follows this contract:

```
python
# Example: python/core/system_scanner/defender.py
import json
import subprocess

def scan() -> dict:
    """
    Returns structured dict with:
    - 'category': str
    - 'status': 'pass' | 'warn' | 'fail' | 'info'
    - 'data': dict (raw extracted data)
    - 'findings': list[dict] (issues found)
    """
    result = {
        "category": "Windows Defender Settings",
        "status": "info",
        "data": {},
        "findings": []
    }
    # ... WMI / Registry / PowerShell queries ...
    return result
```

Module Priority (Critical → Nice-to-have):

Priority	Modules
P0 — Core	`os_info`, `antivirus`, `defender`, `firewall`, `hotfixes`, `uac_policies`, `local_users`, `autorun`
P1 — Important	`amsi_providers`, `audit_policy`, `rdp_connections`, `rdp_sessions`, `rdp_settings`, `ntlm_settings`, `sysmon`, `secure_boot`
P2 — Complete	`dotnet_versions`, `certificates`, `environment`, `file_listing`, `file_info`, `installed products`, `group_policy`, `local_groups`, `ms_updates`, `powershell_history`

Key Implementation Details:

- Use `subprocess.run(["powershell", "-NoProfile", "-Command", ...])` for PowerShell queries
- Use `winreg` stdlib module for direct Registry access (faster than PowerShell for registry)
- Use `wmi` library via COM for WMI queries (antivirus, hotfixes, DNS cache)
- All output as JSON to stdout — Electron reads this pipe
- Each module has error handling that returns `"status": "error"` instead of crashing

Step 3: Python Extraction Engine — Network Scanner

Duration: Day 8-10

Module	Method	Data Extracted
`arp_table.py`	`arp -a` + `Get-NetAdapter`	ARP entries, adapter info
`dns_cache.py`	WMI `Win32_DNSCache`	Cached DNS entries
`network_profiles.py`	`Get-NetConnectionProfile`	Network profile types
`network_shares.py`	`net share` + WMI	Exposed SMB shares
`connections.py`	`Get-NetTCPConnection` + `Get-NetUDPEndpoint`	Active TCP/UDP + PIDs
`rpc_endpoints.py`	`rpcdump` or custom RPC mapper	Mapped RPC endpoints
`open_ports.py`	`python-nmap` or raw socket scan	Open port status
`interfaces.py`	`Get-NetIPConfiguration`	Interface details
`lldp_cdp.py`	Raw packet capture via `scapy`	LLDP/CDP neighbor info
`vlan_devices.py`	ARP sweep + nmap host discovery	Devices on same VLAN
`diagram_generator.py`	`pydot` / `graphviz`	Network topology SVG/PNG

Network Diagram Generation:

```
python
# diagram_generator.py — generates topology from scan results
import pydot

def generate_diagram(interfaces, lldp_neighbors, vlan_devices, output_path):
    graph = pydot.Dot("Network Topology", graph_type="graph", rankdir="TB")
    # Add this machine as central node
    # Add infrastructure devices from LLDP/CDP
    # Add VLAN-attached hosts
    # Add edges with port/protocol labels
    graph.write_svg(output_path)
    return output_path
```

Step 4: Vulnerability Engine & Exploit Crawler

Duration: Day 11-14

Components:

1. `cve_database.py` — Loads the bundled `cve_snapshot.json` (curated Windows CVEs) and queries the NVD REST API v2.0 for online enrichment
2. `cis_benchmarks.py` — Compares system settings against CIS Microsoft Windows 11 Benchmark v1.0 (pre-encoded as JSON rules)
3. `exploit_crawler.py` — Searches Exploit-DB (via `searchsploit` API) and GitHub Advisory Database for public exploits matching detected CVEs
4. `patch_checker.py` — Cross-references detected CVEs against installed hotfixes/KB articles to identify missing patches
5. `severity_scorer.py` — Assigns CVSS v3.1 base scores and translates to Critical/High/Medium/Low/Info

Step 5: Electron + React Frontend

Duration: Day 15-22

Architecture:

```
```

Key UI Components:

Component	Description
`Dashboard`	Landing page with scan buttons, system summary card, last scan timestamp
`SystemScan`	Tabbed view of all 25 system checks with pass/warn/fail badges
`NetworkScan`	Table of connections, shares, open ports + interactive network diagram
`VulnReport`	Severity-sorted vulnerability list with exploit links and patch status
`NetworkDiagram`	D3.js force-directed graph of network topology
`ReportExport`	HTML/PDF export controls with template selection

IPC Contract (preload.js):

```
javascript
// preload.js
const { contextBridge, ipcRenderer } = require('electron');

contextBridge.exposeInMainWorld('agentZero', {
  // Scan operations
  startSystemScan: () => ipcRenderer.invoke('scan:system'),
  startNetworkScan: (options) => ipcRenderer.invoke('scan:network', options),
  startVulnScan: () => ipcRenderer.invoke('scan:vulnerabilities'),
  startFullScan: () => ipcRenderer.invoke('scan:full'),

  // Report operations
  generateReport: (format, data) => ipcRenderer.invoke('report:generate', format,
data),

  // Progress events
  onScanProgress: (callback) => ipcRenderer.on('scan:progress', (_, data) =>
callback(data)),

  // System info
  getLastScanResults: () => ipcRenderer.invoke('results:last'),
});
```

Progress Handling:

- Python engine emits progress JSON lines to stderr: `{"progress": 45, "module": "firewall", "status": "scanning"}`
- Electron main process reads stderr stream and forwards to renderer via `webContents.send('scan:progress', data)`
- React displays animated progress bar with current module name

Step 6: Report Generation, Packaging & Distribution

Duration: Day 23-28

Report Generation:

- HTML report via Jinja2 templates with embedded Chart.js visualizations
- PDF generated from HTML using Puppeteer's `page.pdf()` (Electron already bundles Chromium)
- Report includes: Executive Summary, System Findings, Network Findings, Vulnerability Matrix, Network Diagram, Remediation Steps

Packaging Pipeline:

PyInstaller Configuration:

```
pyinstaller --onefile --clean --name scanner \
  --add-data "data;data" \
  --add-data "report/templates;report/templates" \
  --hidden-import wmi \
  --hidden-import scapy \
  python/main.py
```

electron-builder Configuration:

```
json
{
  "appId": "com.agentzero.scanner",
  "productName": "AgentZero",
  "win": {
    "target": ["nsis", "portable"],
    "icon": "assets/icon.ico",
    "requestedExecutionLevel": "requireAdministrator"
  },
  "extraResources": [
    { "from": "python/dist/scanner.exe", "to": "scanner.exe" }
  ]
}
```

Technology Stack Summary

Layer	Technology	Purpose
Frontend	React 18 + Vite	Dashboard UI
Desktop Shell	Electron 28+	Window management, IPC, OS integration
UI Components	Tailwind CSS + shadcn/ui	Consistent, modern UI
Charts	Chart.js / Recharts	Vulnerability visualizations
Network Diagram	D3.js (force-directed)	Interactive topology map
Backend	Python 3.11+	System/network scanning engine
System Queries	WMI, winreg, subprocess (PowerShell)	Windows OS data extraction
Network Scanning	scapy, python-nmap, psutil	Network reconnaissance
Vuln Database	NVD API v2.0, Exploit-DB	CVE lookup & exploit search
Report Gen	Jinja2 + Puppeteer (PDF)	HTML/PDF report output
Python Packaging	PyInstaller	Standalone .exe backend
App Packaging	electron-builder	Windows installer/portable

Timeline Overview

Risk Register

Risk	Impact	Mitigation
AV flagging PyInstaller .exe	High	Code-sign the binary; use `--clean` flag; whitelist during testing
WMI queries fail without admin	Critical	UAC elevation at launch; graceful degradation for non-admin mode
scapy requires Npcap on Windows	Medium	Bundle Npcap silent installer or fall back to `netstat`-based scanning
NVD API rate limits (5 req/30s)	Low	Use offline CVE snapshot; batch and throttle API calls
Large scan time (>5 min)	Medium	Async module execution; progress UI; allow selective scans
Cross-platform dev friction	Medium	Docker + Windows VM; CI/CD with GitHub Actions Windows runner

> NOTE
>
> **No commits will be made** until you explicitly approve. Even then, I will ask for your confirmation again before pushing anything.

> IMPORTANT
>
> **Next Step:** Once you approve this plan, I'll begin Step 1 — creating the full project scaffolding with all directories, config files, and boilerplate code.